

Week 2 – Day 1 – Home Assignment

1. Using ICEPOT Framework, Generate a Selenium Java test script using TestNG for the Flikart Search functionality

Prompt:

Instructions:

- Generate a Selenium Java test script using TestNG for the Flikart Search functionality
- [STRICT] Use Page Object Model design pattern
- [MANDATORY] @BeforeMethod/@AfterMethod for WebDriver setup/teardown
- [MANDATORY] Explicit waits (WebDriverWait) - no Thread.sleep
- [CRITICAL] Cover both positive and negative test scenarios
- TestNG assertions with clear failure messages

Context:

E-commerce web app Login page — username/password fields, "Remember me" checkbox, "Forgot password" link; once login successfully search a product; runs on Chrome, must support headless execution via TestNG XML.

Examples:

@Test: testValidLogin()

Enter valid username & password, click Login

Assert redirect to /dashboard URL

Assert welcome banner text contains logged-in user's name

@Test: testSearchProduct()

Enter Productname, Click on search Icon

Assert displays the search results

Persona:

- Generate: SDET skilled in Selenium WebDriver, Java, TestNG, and Page Object Model
- Review: Automation Lead, reviewing code quality and framework standards

Output Format:

Complete Java classes — Page Object class + TestNG test class, ready to paste into an IDE, plus a TestNG XML snippet for execution.

Tone:

[STRICT] Clean, production-quality code with meaningful comments.

2. Regression Prioritization:

Context:

You are given a list of 50 test cases from an application. The goal is to identify the highest-risk regression tests that should be executed first after a new release or code change.

Instructions:

1. Review all 50 test cases.
2. Identify the top 10 highest-risk regression tests to run first.
3. Prioritize test cases based on:
 - Module criticality
 - Recent defect history
 - Business impact
 - Failure likelihood
 - Test coverage importance
4. Assign a risk score to each selected test case.
5. Rank the test cases from highest to lowest priority.
6. Provide a brief justification for each ranking.
7. Do not invent test cases or assume defect history that is not provided.
8. If required information is missing, clearly state the assumption or limitation.

Example:

Rank test cases by module criticality and recent defect history.

For example, a payment transfer test with recent production defects should rank higher than a low-impact profile update test.

Input:TC ID	Module	Description	Recent Defects (last 2 releases)	
TC-001	Auth	User login with valid credentials	0	
TC-002	Auth	Login with invalid password shows error	0	
TC-003	Auth	Password reset via email link	1	
TC-004	Auth	Session timeout after 15 min inactivity	0	
TC-005	Auth	Auto-logout on multiple failed login attempts	0	
TC-006	Auth	Login with SSO (Google)	1	
TC-007	Auth	Remember-me checkbox persists session	0	
TC-008	Cart	Add single item to cart	0	
TC-009	Cart	Add multiple items to cart	1	
TC-010	Cart	Remove item from cart	0	
TC-011	Cart	Update item quantity in cart	0	
TC-012	Cart	Cart persists after logout/login	0	
TC-013	Cart	Cart shows correct subtotal	1	
TC-014	Checkout	Checkout with saved address	0	
TC-015	Checkout	Checkout with new address	0	

TC-016	Checkout	Apply valid discount code	2	
TC-017	Checkout	Apply expired discount code shows error	1	
TC-018	Checkout	Checkout with out-of-stock item blocked	0	
TC-019	Checkout	Order summary matches cart before payment	1	
TC-020	Payment	Payment via credit card success	2	
TC-021	Payment	Payment via credit card declined	1	
TC-022	Payment	Payment gateway timeout handling	3	
TC-023	Payment	Refund processed after order cancellation	2	
TC-024	Payment	Partial refund for partial return	1	
TC-025	Payment	Fund transfer between two same-bank accounts	1	
TC-026	Payment	Fund transfer with insufficient balance blocked	0	
TC-027	Order	Order confirmation email sent	0	
TC-028	Order	Order status updates (placed → shipped → delivered)	1	
TC-029	Order	Order cancellation before shipping	1	
TC-030	Order	Order cancellation after shipping blocked	0	
TC-031	Order	Reorder from order history	0	
TC-032	Search	Search by product name	0	
TC-033	Search	Search with filters (category, price)	2	
TC-034	Search	Search with no results shows empty state	0	
TC-035	Search	Search result pagination	0	
TC-036	Search	Search sort by price/rating	1	
TC-037	Profile	Update email address	0	
TC-038	Profile	Update phone number	0	
TC-039	Profile	Change password	0	
TC-040	Profile	Upload profile picture	0	
TC-041	Profile	Delete account	0	
TC-042	Notifications	Order status push notification	1	
TC-043	Notifications	Promotional email opt-out respected	0	
TC-044	Notifications	SMS OTP delivery on login	1	
TC-045	Admin	Admin can view all orders	0	
TC-046	Admin	Admin can issue manual refund	1	
TC-047	Admin	Admin role-based access control	0	
TC-048	API	Public product API returns correct schema	1	
TC-049	API	Rate limiting on public API enforced	0	
TC-050	Reports	Daily sales report export (CSV)	0	

Output Format:

Generate a ranked table with the following columns:

Rank | TC_ID | Test Case Description | Module | Risk Score | Priority | Justification

Persona:

Analyze the test cases as a QA Automation Lead responsible for selecting the most critical regression tests.

Tone:

Concise, technical, and professional.

3. Sprint Retrospective Summary:

Act as a Scrum Master experienced in facilitating sprint retrospectives, identifying improvement opportunities, and creating actionable team plans.

Instructions:

1. Review all provided retrospective notes before preparing the summary.
2. Organize the findings into exactly three sections:
 - **Highlights**
 - **Challenges**
 - **Action Items**
3. Identify recurring themes, significant observations, and patterns across the discussion.
4. Summarize what went well without adding information that is not present in the notes.
5. Convert relevant challenges into clear, actionable improvement items.
6. Assign an owner and due date only when explicitly provided in the notes.
7. If an owner or due date is missing, write "**Not specified.**"
8. Include success criteria for each action item based on the expected improvement described in the notes.
9. Do not invent owners, due dates, success criteria, or other information.
10. Keep the summary concise, constructive, blame-free, and professional.
11. Avoid repeating the same observation across multiple sections unless necessary for clarity.

Context:

You are given notes from a 2-week sprint retrospective involving 8 team members. The notes may include what went well, what did not go well, recurring themes, and proposed improvement actions.

Example Transformation:**Retrospective Note:**

"We kept blocking on the staging environment."

Action Item:

"Establish a clear process for resolving staging environment blockers."

Input:

[Paste the complete 2-week sprint retrospective notes here]

Output Format:

1. Highlights

- Summarize key things that went well.
- Include notable achievements, positive team collaboration, and successful practices.

2. Challenges

- Summarize key problems, blockers, recurring themes, and improvement areas.
- Focus on facts and observations rather than assigning blame.

3. Action Items

Action Item	Owner	Due Date	Success Criteria
[Clear, actionable improvement item]	[Name or Not specified]	[Date or Not specified]	[Expected measurable or observable improvement]

Persona:

Analyze and summarize the retrospective as a Scrum Master facilitating the team discussion.

Tone:

Constructive, blame-free, concise, and professional.

4. Interactive CoT — Investigate an Automation Failure

Act as a Test Automation Engineer. Investigate why my test passes locally but fails in CI.

Ask 5 questions, ONE AT A TIME, based on my answers — request code/logs/screenshots when useful. Wait for each answer.

Then give a table: Cause | Evidence | Verification | Fix. Mark confirmed vs. hypothesis — no cause without evidence.

START WITH QUESTION 1 ONLY.

Root-Cause Analysis				Upgrade	Share
Cause	Evidence	Verification	Fix		
Headless-specific behavior — Hypothesis	Jenkins runs Chrome headless; local execution may differ.	Run the same test locally in headless mode with the same Chrome options. Compare the failing step.	Keep headless configuration consistent; validate element visibility and interaction behavior.		
Timing or synchronization issue — Hypothesis	A 10-second explicit wait is configured, but the actual wait condition and failure step are unknown.	Review the failing line and replace fixed assumptions with waits for the required state, such as visibility, clickability, or URL.	Use condition-based <code>WebDriverWait</code> ; avoid <code>Thread.sleep</code> .		
Chrome / ChromeDriver version mismatch — Hypothesis	CI and local browser environments may differ. No versions were provided.	Print Chrome and ChromeDriver versions in both environments and compare them.	Pin compatible browser/driver versions or use a controlled browser image.		
Jenkins environment difference — Hypothesis	CI runs under Jenkins, potentially with different OS, permissions, display, or filesystem behavior.	Compare OS, Java version, browser version, environment variables, and execution user.	Standardize the CI environment and configure required permissions/dependencies.		
Incorrect locator or missing element — Hypothesis	No failing test step or exception was provided	Capture the failing locator, page URL, screenshot, and page source at failure.	Correct locators or add appropriate waits after confirming the actual failure.		

